JAVA SE · PROGRAMMING REFERENCE

Core Java

The Complete Reference Guide

All Core Topics Explained in Detail with Examples

From fundamentals and object-oriented programming to collections, generics, lambdas, streams, multithreading and I/O

Preface

This guide is a complete, example-driven reference to **Core Java** — the part of the Java platform every developer must master before moving on to frameworks such as Spring, Hibernate or Android. It covers the language from the ground up: syntax and data types, control flow, object-oriented programming, exception handling, the Collections Framework, generics, the functional features introduced in Java 8 (lambdas and streams), multithreading, and file input/output.

Every concept is explained in plain language and then demonstrated with a short, runnable code example so you can see exactly how it behaves. Comparison tables summarise the differences that interviewers and code reviewers care about most.

How to use this guide

- **Read sequentially** if you are new — each part builds on the previous one.
- **Jump via the Table of Contents** (clickable) if you need a refresher on a single topic.
- **Type out the examples** yourself. Reading code is not the same as writing it.
- All examples target **Java SE 8 or later**; notes call out features added in newer versions.

Conventions: code and identifiers appear in a monospace font on a shaded background. Class names use UpperCamelCase; variables and methods use lowerCamelCase, following standard Java style.

Table of Contents

PART 1 Getting Started with Java 6

1.1 What Is Java? 6
1.2 JDK, JRE and JVM 6
1.3 How Java Code Runs 6
1.4 Your First Program 7
1.5 Comments and Naming Conventions 7

PART 2 Variables, Data Types and Operators 8

2.1 Variables and Identifiers 8
2.2 Primitive Data Types 8
2.3 Reference Types and Literals 8
2.4 Type Casting 8
2.5 Operators 9
2.6 Reading Input with Scanner 10

PART 3 Control Flow Statements 11

3.1 if, if-else and the else-if Ladder 11
3.2 The switch Statement 11
3.3 for Loop and Enhanced for 11
3.4 while and do-while 11
3.5 break, continue and Labels 12

PART 4 Arrays and Strings 13

4.1 Arrays 13
4.2 Multidimensional and Jagged Arrays 13
4.3 The Arrays Utility Class 13
4.4 The String Class and Immutability 13
4.5 Common String Methods 14
4.6 StringBuilder and StringBuffer 14

PART 5 Object-Oriented Programming (OOP) 15

5.1 Classes and Objects 15
5.2 Methods and Method Overloading 15
5.3 Constructors and Constructor Chaining 15
5.4 The this Keyword 16
5.5 Inheritance 16
5.6 Polymorphism 16
5.7 Encapsulation 17
5.8 Abstraction with Abstract Classes 17
5.9 Abstraction with Interfaces 17
5.10 Abstract Class vs Interface 18
5.11 Access Modifiers 18
5.12 The static Keyword 18

5.13 The final Keyword 19
5.14 The Object Class: toString, equals, hashCode 19
5.15 Packages and import 19

PART 6 Exception Handling 21

6.1 What Are Exceptions? 21
6.2 try-catch-finally 21
6.3 Checked vs Unchecked Exceptions 21
6.4 throw vs throws 21
6.5 try-with-resources 21
6.6 Custom Exceptions 22

PART 7 More Language Features 23

7.1 Wrapper Classes, Autoboxing and Unboxing 23
7.2 Enums 23
7.3 Inner (Nested) Classes 23
7.4 Varargs (Variable Arguments) 24
7.5 Annotations 24

PART 8 Collections Framework and Generics 25

8.1 Overview of the Collections Framework 25
8.2 Generics 25
8.3 List Implementations 25
8.4 Set Implementations 26
8.5 Map Implementations 26
8.6 Queue and Deque 26
8.7 Iterator and ListIterator 27
8.8 Comparable vs Comparator 27
8.9 The Collections Utility Class 27

PART 9 Java 8 Functional Features 28

9.1 Lambda Expressions 28
9.2 Functional Interfaces 28
9.3 Method References 28
9.4 The Stream API 29
9.5 Optional 29

PART 10 Multithreading and Concurrency 30

10.1 Threads: Process vs Thread 30
10.2 Thread Life Cycle 30
10.3 Common Thread Methods 30
10.4 Synchronization 31
10.5 Inter-thread Communication 31

PART 11 Input / Output and File Handling 32

11.1 The File Class 32
11.2 Byte Streams 32

11.3 Character Streams 32
11.4 Serialization 32
11.5 Modern File I/O (java.nio) 33

PART A Appendix 34

A.1 Java Keywords (Reserved Words) 34
A.2 Operator Precedence (high → low) 34
A.3 Common Pitfalls and Best Practices 34
A.4 Quick Glossary 35

PART 1 Getting Started with Java

1.1 What Is Java?

Java is a high-level, class-based, **object-oriented** programming language created by James Gosling at Sun Microsystems and released in 1995 (now owned by Oracle). Its guiding promise is **"Write Once, Run Anywhere" (WORA)**: source code is compiled to platform-neutral bytecode that runs on any device equipped with a Java Virtual Machine.

Key features:

- **Platform independent** — bytecode runs on any JVM, regardless of OS or hardware.
- **Object-oriented** — everything is organised around classes and objects.
- **Simple & familiar** — C/C++-like syntax without pointers or manual memory management.
- **Robust** — strong type checking, exception handling, and automatic garbage collection.
- **Secure** — runs inside the JVM sandbox; no direct memory access.
- **Multithreaded** — built-in support for concurrent programming.
- **Portable & high performance** — JIT compilation turns hot bytecode into native code.

Editions of Java

- **Java SE** (Standard Edition) — the core language and APIs. This guide covers Java SE.
- **Java EE / Jakarta EE** (Enterprise Edition) — web and enterprise APIs (Servlets, JPA…).
- **Java ME** (Micro Edition) — for resource-constrained embedded devices.

1.2 JDK, JRE and JVM

These three acronyms are central and frequently confused. The JDK contains the JRE, and the JRE contains the JVM.

Term	Stands for	What it contains / does
JVM	Java Virtual Machine	An abstract machine that **executes bytecode**. It loads classes, verifies bytecode, and runs it (interpreting or JIT-compiling). It is platform specific.
JRE	Java Runtime Environment	The **JVM + core libraries** needed to *run* Java programs. No compiler.
JDK	Java Development Kit	The **JRE + development tools** (compiler `javac`, `java`, `jar`, debugger). You need the JDK to *build* programs.

Rule of thumb: install the JDK to develop; the JRE alone is enough only to run.

1.3 How Java Code Runs

Java is both compiled and interpreted. The flow from source to output is:

- **1. Write** source code in a `.java` file.
- **2. Compile** with `javac` → produces `.class` files containing **bytecode**.
- **3. Run** with `java` → the JVM loads the bytecode, verifies it, and executes it.
- **4. JIT** (Just-In-Time) compiler converts frequently used ("hot") bytecode into native machine code for speed.

Because the `.class` bytecode is identical on every platform, the same compiled program runs unchanged on Windows, macOS or Linux — only the JVM underneath differs.

1.4 Your First Program

The classic starting point prints a line of text to the console. Save it as `HelloWorld.java` — the file name must match the public class name.

```
public class HelloWorld {
    // Execution always begins in the main method.
    public static void main(String[] args) {
        System.out.println("Hello, World!");
    }
}
```

Compile and run from a terminal:

```
javac HelloWorld.java     // produces HelloWorld.class
java HelloWorld           // prints: Hello, World!
```

Anatomy of the main method

- `public` — the JVM must access it from outside the class.
- `static` — runs without creating an object of the class.
- `void` — returns nothing.
- `main` — the fixed name the JVM looks for as the entry point.
- `String[] args` — receives command-line arguments.
- `System.out.println(...)` — prints a line; `print` omits the newline.

1.5 Comments and Naming Conventions

Comments document code and are ignored by the compiler.

```
// single-line comment

/* multi-line
   comment */

/**
 * Javadoc comment - used to generate API docs.
 * @param name the user's name
 */
```

Widely followed naming conventions (not enforced by the compiler, but expected by every Java team):

Element	Convention	Example
Class / Interface	UpperCamelCase	`BankAccount, Runnable`
Method / variable	lowerCamelCase	`getBalance, totalCount`
Constant	UPPER_SNAKE_CASE	`MAX_SIZE, PI`
Package	all lowercase, dotted	`com.example.app`

PART 2 Variables, Data Types and Operators

2.1 Variables and Identifiers

A **variable** is a named container for a value. Java is **statically typed**: every variable has a fixed type declared before use. An **identifier** is the variable's name; it may contain letters, digits, _ and $, but cannot start with a digit or be a reserved keyword.

```
int age = 25;            // declaration + initialization
double price;            // declaration
price = 19.99;           // assignment
final int MAX = 100;     // constant - cannot be reassigned
```

Kinds of variables

- **Local** — declared inside a method/block; no default value, must be initialized before use.
- **Instance** — declared in a class, one copy per object; gets a default value (0, null, false).
- **Static** — declared `static`; one copy shared by all objects.

2.2 Primitive Data Types

Java has eight built-in **primitive** types. They hold simple values directly (not objects) and have fixed sizes on every platform.

Type	Size	Range / values	Default	Example
byte	8-bit	–128 to 127	0	`byte b = 100;`
short	16-bit	–32,768 to 32,767	0	`short s = 5000;`
int	32-bit	~ ±2.1 billion	0	`int i = 100000;`
long	64-bit	very large integers	0L	`long l = 99L;`
float	32-bit	~7 decimal digits	0.0f	`float f = 3.14f;`
double	64-bit	~15 decimal digits	0.0d	`double d = 3.14;`
char	16-bit	single Unicode char	'\u0000'	`char c = 'A';`
boolean	1-bit*	true or false	false	`boolean ok = true;`

**The JVM does not precisely define boolean size; conceptually it holds one bit.*

2.3 Reference Types and Literals

Anything that is not a primitive is a **reference type**: classes, interfaces, arrays, enums and `String`. A reference variable stores the memory address of an object (or `null`), not the object itself.

```
String name = "Alice";          // String literal
int[] nums = {1, 2, 3};         // array object
Integer boxed = 42;             // wrapper object

int hex   = 0xFF;               // 255 in hexadecimal
int bin   = 0b1010;             // 10 in binary
long big  = 1_000_000L;         // underscores aid readability
double sci = 1.5e3;             // 1500.0
```

2.4 Type Casting

Converting a value from one type to another is **casting**. **Widening** (small → large) is automatic and safe. **Narrowing** (large → small) must be explicit and may lose data.

```
// Widening - implicit (no data loss)
int i = 100;
double d = i;          // 100.0

// Narrowing - explicit cast required
double pi = 3.99;
int truncated = (int) pi;   // 3 (decimal part dropped)

long l = 130;
byte b = (byte) l;          // -126 (overflow wraps around)
```

Order of widening: `byte` → `short` → `int` → `long` → `float` → `double`. `char` widens to `int` and above.

2.5 Operators

Operators perform operations on variables and values. Java groups them into several categories.

Arithmetic and assignment

Operator	Meaning	Example (a=10, b=3)
+ – * /	add, subtract, multiply, divide	`a / b` → `3 (integer div)`
%	modulus (remainder)	`a % b` → `1`
++ / --	increment / decrement	`a++` → `11`
= += -= *= /=	assignment / compound	`a += 5` → `a = 15`

Relational and logical

Operator	Meaning	Result type
== !=	equal to / not equal to	boolean
> < >= <=	comparison	boolean
&&	logical AND (short-circuit)	boolean
||	logical OR (short-circuit)	boolean
!	logical NOT	boolean

Short-circuit: with && the right side is skipped if the left is `false`; with || it is skipped if the left is `true`. This is used to guard against errors such as null checks.

Bitwise, ternary and instanceof

```
int x = 6, y = 3;
System.out.println(x & y);   // 2  bitwise AND
System.out.println(x | y);   // 7  bitwise OR
System.out.println(x ^ y);   // 5  bitwise XOR
System.out.println(x << 1);  // 12 left shift (x * 2)

// Ternary: condition ? valueIfTrue : valueIfFalse
int max = (x > y) ? x : y;   // 6

// instanceof: tests an object's type
String s = "hi";
boolean isStr = s instanceof String;   // true
```

2.6 Reading Input with Scanner

The `java.util.Scanner` class reads typed input from the console (or any input stream).

```
import java.util.Scanner;

public class InputDemo {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter your name: ");
        String name = sc.nextLine();

        System.out.print("Enter your age: ");
        int age = sc.nextInt();

        System.out.println("Hi " + name + ", age " + age);
        sc.close();
    }
}
```

Common methods: `next()` (one word), `nextLine()` (whole line), `nextInt()`, `nextDouble()`, `nextBoolean()`.

PART 3 Control Flow Statements

3.1 if, if-else and the else-if Ladder

Conditional statements run a block only when a boolean condition is true.

```
int score = 78;

if (score >= 90) {
    System.out.println("Grade A");
} else if (score >= 75) {
    System.out.println("Grade B");   // this runs
} else if (score >= 60) {
    System.out.println("Grade C");
} else {
    System.out.println("Fail");
}
```

3.2 The switch Statement

`switch` chooses among many fixed values of a single variable — cleaner than a long else-if ladder. Without `break`, execution "falls through" to the next case.

```
int day = 3;
switch (day) {
    case 1: System.out.println("Monday");    break;
    case 2: System.out.println("Tuesday");   break;
    case 3: System.out.println("Wednesday"); break;   // matches
    default: System.out.println("Other day");
}
```

Java 14+ adds an **enhanced switch** with arrow labels (no fall-through) that can also return a value:

```
String type = switch (day) {
    case 6, 7 -> "Weekend";
    default   -> "Weekday";
};
```

3.3 for Loop and Enhanced for

The `for` loop is ideal when the number of iterations is known. The **enhanced for** (for-each) iterates over arrays and collections.

```
// Classic for: init ; condition ; update
for (int i = 1; i <= 5; i++) {
    System.out.print(i + " ");      // 1 2 3 4 5
}

// Enhanced for-each
int[] nums = {10, 20, 30};
for (int n : nums) {
    System.out.print(n + " ");      // 10 20 30
}
```

3.4 while and do-while

`while` tests the condition *before* each iteration (may run zero times). `do-while` tests *after*, so the body always runs at least once.

```
int i = 1;
while (i <= 3) {              // pre-test
    System.out.println(i);
```

```
    i++;
}

int j = 1;
do {                          // post-test (runs >= 1 time)
    System.out.println(j);
    j++;
} while (j <= 3);
```

3.5 break, continue and Labels

`break` exits the loop entirely; `continue` skips to the next iteration. A **label** lets you break/continue an outer loop from inside a nested one.

```
for (int i = 1; i <= 5; i++) {
    if (i == 4) break;        // stop the loop at 4
    if (i == 2) continue;     // skip printing 2
    System.out.print(i + " ");   // 1 3
}

outer:
for (int a = 0; a < 3; a++) {
    for (int b = 0; b < 3; b++) {
        if (a + b == 2) break outer;   // exits BOTH loops
    }
}
```

PART 4 Arrays and Strings

4.1 Arrays

An **array** is a fixed-size, ordered collection of elements of the *same type*, stored contiguously and accessed by a zero-based index. Its length is fixed once created.

```
// Declaration + allocation
int[] a = new int[5];          // five zeros
a[0] = 10; a[1] = 20;

// Declaration + initialization
int[] b = {1, 2, 3, 4, 5};

System.out.println(b.length);  // 5  (a field, not a method)
System.out.println(b[2]);      // 3

for (int n : b) System.out.print(n + " ");
```

Accessing an invalid index throws `ArrayIndexOutOfBoundsException`.

4.2 Multidimensional and Jagged Arrays

A two-dimensional array is an “array of arrays” (think rows and columns). Rows may have different lengths — a **jagged** array.

```
int[][] grid = {
    {1, 2, 3},
    {4, 5, 6}
};
System.out.println(grid[1][2]);   // 6

// Jagged: rows of unequal length
int[][] jag = new int[2][];
jag[0] = new int[]{1, 2};
jag[1] = new int[]{3, 4, 5};
```

4.3 The Arrays Utility Class

`java.util.Arrays` provides handy static helpers for sorting, searching, filling and printing arrays.

```
import java.util.Arrays;

int[] a = {5, 2, 8, 1};
Arrays.sort(a);                       // {1, 2, 5, 8}
System.out.println(Arrays.toString(a));
int idx = Arrays.binarySearch(a, 5);  // index of 5 (on sorted array)
int[] copy = Arrays.copyOf(a, 6);     // longer copy, padded with 0
```

4.4 The String Class and Immutability

A `String` is a sequence of characters and one of the most-used classes. Crucially, Strings are **immutable**: once created, their contents never change. Any “modifying” method returns a *new* String.

```
String s1 = "Hello";              // string literal (pooled)
String s2 = new String("Hello");  // explicit new object

String upper = s1.toUpperCase();  // returns NEW string "HELLO"
// s1 is still "Hello"

// == compares references; equals() compares content
System.out.println(s1 == s2);        // false (different objects)
```

```
System.out.println(s1.equals(s2));   // true  (same characters)
```

String pool: identical string *literals* share one object in a special memory pool, which is why two literals compare equal with ==. Always compare string content with `.equals()`, never ==.

4.5 Common String Methods

Frequently used methods (all return new values; the original is unchanged):

Method	Purpose	Example → result
length()	number of characters	`"java".length() → 4`
charAt(i)	char at index i	`"java".charAt(0) → j`
substring(a,b)	chars from a to b–1	`"java".substring(1,3) → av`
indexOf(x)	first index of x (or –1)	`"java".indexOf("v") → 2`
toUpperCase()	upper-case copy	`"java" → JAVA`
trim() / strip()	remove edge whitespace	`" hi " → hi`
replace(a,b)	replace chars/text	`"aaa".replace("a","b") → bbb`
split(regex)	split into String[]	`"a,b".split(",") → [a, b]`
contains(x)	substring test	`"java".contains("av") → true`
equals / equalsIgnoreCase	content comparison	`"A".equalsIgnoreCase("a") → true`

4.6 StringBuilder and StringBuffer

Because Strings are immutable, building a string by repeated + concatenation in a loop is wasteful (each step creates a new object). `StringBuilder` is a **mutable** sequence designed for exactly this.

```
StringBuilder sb = new StringBuilder();
sb.append("Hello");
sb.append(", ").append("World");
sb.insert(0, ">> ");
sb.reverse();
System.out.println(sb.toString());

// Efficient loop building
StringBuilder out = new StringBuilder();
for (int i = 0; i < 5; i++) out.append(i);
System.out.println(out);          // 01234
```

Feature	String	StringBuilder	StringBuffer
Mutable?	No	Yes	Yes
Thread-safe?	Yes (immutable)	No	Yes (synchronized)
Performance	Slow for edits	Fastest	Slower than Builder
Use when…	value is constant	single-thread edits	multi-thread edits

PART 5 Object-Oriented Programming (OOP)

5.1 Classes and Objects

OOP models software as interacting **objects**. A **class** is a blueprint that defines *state* (fields) and *behaviour* (methods); an **object** is a concrete instance created from that blueprint with `new`.

```
class Car {
    String brand;          // field (state)
    int speed;

    void accelerate(int by) {   // method (behaviour)
        speed += by;
    }
}

public class Demo {
    public static void main(String[] args) {
        Car c = new Car();      // create an object
        c.brand = "Toyota";
        c.accelerate(40);
        System.out.println(c.brand + " @ " + c.speed);
    }
}
```

The four pillars of OOP

- **Encapsulation** — bundle data with the methods that operate on it; hide internals.
- **Inheritance** — derive a new class from an existing one to reuse behaviour.
- **Polymorphism** — one interface, many implementations (“many forms”).
- **Abstraction** — expose essential features, hide complex detail.

5.2 Methods and Method Overloading

A **method** is a named block of reusable code with an optional return value and parameters. **Overloading** means defining several methods with the *same name* but *different parameter lists*; the compiler picks the right one by the arguments (compile-time polymorphism).

```
class MathUtil {
    int add(int a, int b)           { return a + b; }
    double add(double a, double b)  { return a + b; }   // overload
    int add(int a, int b, int c)    { return a + b + c; }
}
// MathUtil m = new MathUtil();
// m.add(2, 3);        -> 5   (calls int version)
// m.add(2.5, 1.5);    -> 4.0 (calls double version)
```

Overloading is decided by **number, types or order** of parameters — *not* by the return type alone.

5.3 Constructors and Constructor Chaining

A **constructor** initialises a new object. It shares the class name, has no return type, and runs automatically on `new`. If you write none, Java supplies a no-arg **default constructor**.

```
class Student {
    String name;
    int age;

    Student() {                     // no-arg constructor
        this("Unknown", 0);         // calls the other via this()
```

```
    }
    Student(String name, int age) {   // parameterized
        this.name = name;             // this -> current object
        this.age = age;
    }
}
```

- `this(...)` calls another constructor in the *same* class.
- `super(...)` calls a constructor of the *parent* class.
- A constructor call via `this()/super()` must be the first statement.

5.4 The this Keyword

`this` is a reference to the **current object**. It is used to disambiguate fields from parameters of the same name, to pass the current object to another method, and to chain constructors.

```
class Account {
    double balance;
    Account deposit(double balance) {
        this.balance += balance;   // this.balance = field; balance = param
        return this;               // enables method chaining
    }
}
// new Account().deposit(100).deposit(50);
```

5.5 Inheritance

Inheritance lets a *subclass* acquire the fields and methods of a *superclass* using `extends`, promoting code reuse and an “is-a” relationship. `super` accesses parent members.

```
class Animal {
    void eat() { System.out.println("eating"); }
}

class Dog extends Animal {        // Dog IS-A Animal
    void bark() { System.out.println("woof"); }
    void show() {
        eat();                    // inherited from Animal
        bark();
    }
}
```

Types of inheritance in Java

- **Single** — one subclass, one superclass.
- **Multilevel** — a chain: C extends B extends A.
- **Hierarchical** — several subclasses share one superclass.
- **Multiple inheritance of classes is NOT allowed** (to avoid the diamond problem) — but a class may implement multiple *interfaces*.

5.6 Polymorphism

Polymorphism means a single action behaves differently in different contexts. Java has two forms.

Aspect	Compile-time (Overloading)	Runtime (Overriding)
Resolved at	compile time	run time
Mechanism	same name, different params	subclass redefines parent method
Also called	static / early binding	dynamic / late binding

Aspect	Compile-time (Overloading)	Runtime (Overriding)
Signature	must differ	must be identical

Method overriding: a subclass provides its own version of an inherited method (same signature). The actual method that runs is chosen by the object's real type at runtime — the heart of dynamic dispatch.

```
class Shape { double area() { return 0; } }

class Circle extends Shape {
    double r;
    Circle(double r) { this.r = r; }
    @Override
    double area() { return Math.PI * r * r; }   // overrides
}

Shape s = new Circle(2);     // parent reference, child object
System.out.println(s.area()); // 12.566... (Circle's version runs)
```

The `@Override` annotation is optional but recommended: the compiler verifies you really are overriding a method.

5.7 Encapsulation

Encapsulation keeps fields `private` and exposes controlled access through public **getter** and **setter** methods. This protects the object's state and lets you add validation.

```
class BankAccount {
    private double balance;            // hidden

    public double getBalance() {       // getter
        return balance;
    }
    public void deposit(double amt) {  // controlled setter
        if (amt > 0) balance += amt;   // validation
    }
}
```

5.8 Abstraction with Abstract Classes

Abstraction exposes *what* an object does while hiding *how*. An `abstract` class cannot be instantiated and may contain **abstract methods** (no body) that subclasses must implement, alongside concrete methods.

```
abstract class Shape {
    abstract double area();          // no body - must be overridden

    void describe() {                // concrete method
        System.out.println("Area = " + area());
    }
}

class Square extends Shape {
    double side;
    Square(double s) { side = s; }
    double area() { return side * side; }
}
```

5.9 Abstraction with Interfaces

An **interface** is a fully abstract contract: a set of method signatures a class promises to implement via `implements`. A class may implement *many* interfaces, which is how Java achieves multiple inheritance of type.

```
interface Drawable {
    void draw();                        // public abstract by default

    default void hint() {               // default method (Java 8+)
        System.out.println("Drawing...");
    }
    static int sides() { return 0; }    // static method (Java 8+)
}

class Triangle implements Drawable {
    public void draw() {
        System.out.println("Drawing a triangle");
    }
}
```

- Interface fields are implicitly `public static final` (constants).
- Methods are `public abstract` unless marked `default`, `static` or `private`.
- **default methods** (Java 8) let interfaces evolve without breaking implementers.

5.10 Abstract Class vs Interface

Both model abstraction; choosing between them is a classic design decision.

Aspect	Abstract class	Interface
Keyword	extends	implements
Multiple inheritance	No (one superclass)	Yes (many interfaces)
Fields	any kind	public static final only
Methods	abstract + concrete	abstract + default/static (8+)
Constructors	Yes	No
Use when…	shared code + state, "is-a"	pure contract, "can-do"

5.11 Access Modifiers

Access modifiers control the **visibility** of classes, fields and methods.

Modifier	Same class	Same package	Subclass (other pkg)	Everywhere
private	Yes	No	No	No
default (none)	Yes	Yes	No	No
protected	Yes	Yes	Yes	No
public	Yes	Yes	Yes	Yes

“default” (also called package-private) applies when no modifier is written.

5.12 The static Keyword

`static` members belong to the **class itself** rather than to any instance. There is exactly one copy, shared by all objects, and they are accessed via the class name.

```
class Counter {
    static int count = 0;     // shared across all objects
    int id;

    Counter() { id = ++count; }
```

```
    static void report() {    // static method - no 'this'
        System.out.println("Total: " + count);
    }

    static { System.out.println("Class loaded"); }  // static block
}
// new Counter(); new Counter();
// Counter.report();   -> Total: 2
```

- **static variable** — one shared copy (e.g. a counter or constant).
- **static method** — callable without an object; cannot use `this` or instance fields directly.
- **static block** — runs once when the class is first loaded; used for setup.

5.13 The final Keyword

`final` means “cannot change”, with a precise meaning depending on what it modifies.

Applied to	Effect
final variable	value can be assigned once — becomes a constant
final method	cannot be overridden by a subclass
final class	cannot be extended (e.g. `String`)
final parameter	cannot be reassigned inside the method

```
final double PI = 3.14159;     // constant
// PI = 3.14;  // COMPILE ERROR

final class Constants { }      // cannot be subclassed
```

5.14 The Object Class: toString, equals, hashCode

Every class implicitly extends `java.lang.Object`, inheriting methods commonly overridden to give objects meaningful behaviour.

```
class Point {
    int x, y;
    Point(int x, int y) { this.x = x; this.y = y; }

    @Override
    public String toString() {     // readable text form
        return "Point(" + x + ", " + y + ")";
    }
    @Override
    public boolean equals(Object o) {   // logical equality
        if (!(o instanceof Point)) return false;
        Point p = (Point) o;
        return x == p.x && y == p.y;
    }
    @Override
    public int hashCode() { return x * 31 + y; }
}
```

Contract: if two objects are `equals()`, they must return the same `hashCode()`. Violating this breaks hash-based collections like `HashMap` and `HashSet`.

5.15 Packages and import

A **package** is a namespace that groups related classes, prevents naming clashes, and supports access control. Declare it as the first line of a file; use `import` to access classes from other packages.

```
package com.example.bank;     // must be the first statement

import java.util.List;        // import a single class
import java.util.*;           // import all of a package

public class Account { /* ... */ }
```

Classes in `java.lang` (e.g. `String`, `Math`, `System`) are imported automatically.

PART 6 Exception Handling

6.1 What Are Exceptions?

An **exception** is an event that disrupts the normal flow of a program at runtime, such as dividing by zero or opening a missing file. Java represents these as objects in a class hierarchy rooted at `Throwable`.

- `Throwable` → the root of all errors and exceptions.
- `Error` — serious problems the app should not catch (e.g. `OutOfMemoryError`).
- `Exception` — conditions a program can reasonably handle.
- `RuntimeException` — unchecked exceptions (programming bugs).

6.2 try-catch-finally

Wrap risky code in `try`; handle failures in `catch`; put cleanup that must always run in `finally`.

```
try {
    int[] a = {1, 2, 3};
    System.out.println(a[5]);       // throws exception
} catch (ArrayIndexOutOfBoundsException e) {
    System.out.println("Bad index: " + e.getMessage());
} catch (Exception e) {             // more general - must come AFTER
    System.out.println("Other error");
} finally {
    System.out.println("Always runs (cleanup)");
}
```

Order matters: catch more specific exception types before more general ones. A single catch can handle several types with the pipe: `catch (IOException | SQLException e)`.

6.3 Checked vs Unchecked Exceptions

Java distinguishes two categories, which affects whether the compiler forces you to handle them.

Aspect	Checked	Unchecked
Checked by compiler?	Yes — must handle or declare	No
Superclass	Exception (not RuntimeException)	RuntimeException
Typical cause	external/recoverable conditions	programming bugs
Examples	IOException, SQLException	NullPointer, ArithmeticException

6.4 throw vs throws

`throw` actually raises an exception object; `throws` declares in a method signature that it might raise one, delegating handling to the caller.

```
static void setAge(int age) throws IllegalArgumentException {
    if (age < 0) {
        throw new IllegalArgumentException("Age cannot be negative");
    }
    System.out.println("Age set to " + age);
}
```

6.5 try-with-resources

Resources such as files and streams must be closed. **try-with-resources** (Java 7+) declares them in parentheses and closes them automatically — even if an exception occurs. The resource must implement `AutoCloseable`.

```
try (Scanner sc = new Scanner(new File("data.txt"))) {
    while (sc.hasNextLine()) {
        System.out.println(sc.nextLine());
    }
}   // sc.close() is called automatically here
catch (IOException e) {
    System.out.println("Cannot read file");
}
```

6.6 Custom Exceptions

Define your own exception by extending `Exception` (checked) or `RuntimeException` (unchecked) to model domain-specific errors clearly.

```
class InsufficientFundsException extends Exception {
    public InsufficientFundsException(String msg) {
        super(msg);
    }
}

void withdraw(double amt, double balance) throws InsufficientFundsException {
    if (amt > balance) {
        throw new InsufficientFundsException("Not enough balance");
    }
}
```

PART 7 More Language Features

7.1 Wrapper Classes, Autoboxing and Unboxing

Each primitive type has a corresponding **wrapper class** (an object form). Wrappers are required wherever objects are needed — most importantly inside collections, which cannot hold primitives.

Primitive	Wrapper	Primitive	Wrapper
byte	Byte	long	Long
short	Short	float	Float
int	Integer	double	Double
char	Character	boolean	Boolean

Autoboxing automatically converts a primitive to its wrapper; **unboxing** does the reverse.

```
Integer boxed = 42;          // autoboxing  (int -> Integer)
int prim = boxed;            // unboxing    (Integer -> int)

List<Integer> list = new ArrayList<>();
list.add(10);                // int autoboxed to Integer

int parsed = Integer.parseInt("123");   // String -> int
String text = Integer.toString(123);    // int -> String
```

7.2 Enums

An **enum** defines a fixed set of named constants in a type-safe way. Java enums are full classes: they can have fields, constructors and methods.

```
enum Day { MON, TUE, WED, THU, FRI, SAT, SUN }

enum Planet {
    EARTH(9.81), MARS(3.71);          // constants with data
    private final double gravity;
    Planet(double g) { gravity = g; }
    double getGravity() { return gravity; }
}

Day d = Day.WED;
switch (d) {
    case WED -> System.out.println("Midweek");
    default  -> System.out.println("Other");
}
```

Useful methods: `values()` (array of all constants), `ordinal()` (position), `name()` and `valueOf("X")`.

7.3 Inner (Nested) Classes

A class defined inside another class is a **nested class**. There are four kinds, each suited to a different need.

- **Member inner class** — non-static, tied to an outer instance; can access its private members.
- **Static nested class** — declared `static`; no link to an outer instance.
- **Local class** — declared inside a method.
- **Anonymous class** — an unnamed class created and instantiated in one expression, often to implement an interface on the spot.

```
interface Greeter { void greet(); }
```

```
public class Outer {
    static class Engine { }          // static nested

    void run() {
        // anonymous class implementing Greeter
        Greeter g = new Greeter() {
            public void greet() {
                System.out.println("Hi!");
            }
        };
        g.greet();
    }
}
```

7.4 Varargs (Variable Arguments)

Varargs let a method accept any number of arguments of a type, written with `...`. Inside the method the parameter behaves as an array. It must be the *last* parameter.

```
static int sum(int... numbers) {     // 0..n ints
    int total = 0;
    for (int n : numbers) total += n;
    return total;
}
// sum()          -> 0
// sum(1, 2, 3)   -> 6
// sum(5, 5)      -> 10
```

7.5 Annotations

Annotations are metadata attached to code. They do not change logic directly but can be read by the compiler or at runtime by frameworks.

- `@Override` — declares a method overrides a superclass method.
- `@Deprecated` — marks an element as obsolete.
- `@SuppressWarnings` — tells the compiler to hide specific warnings.
- `@FunctionalInterface` — enforces a single abstract method.

PART 8 Collections Framework and Generics

8.1 Overview of the Collections Framework

The **Collections Framework** (`java.util`) is a set of reusable data-structure classes and interfaces for storing and manipulating groups of objects. The core interfaces are `Collection` (root), `List`, `Set`, `Queue` and — separately — `Map`.

Interface	Allows duplicates?	Ordered?	Typical implementations
List	Yes	Yes (by index)	ArrayList, LinkedList
Set	No	Depends	HashSet, LinkedHashSet, TreeSet
Queue	Yes	FIFO	LinkedList, PriorityQueue
Map (key→value)	No duplicate keys	Depends	HashMap, TreeMap, LinkedHashMap

8.2 Generics

Generics let classes and methods operate on a type specified as a parameter, giving **compile-time type safety** and removing casts. The diamond <> holds the type argument.

```
// Without generics (old, unsafe):
// List list = new ArrayList();
// list.add("x"); String s = (String) list.get(0);  // cast needed

// With generics (safe):
List<String> names = new ArrayList<>();
names.add("Alice");
String s = names.get(0);     // no cast; compiler guarantees String

// A generic class
class Box<T> {
    private T value;
    void set(T v) { value = v; }
    T get()       { return value; }
}
Box<Integer> b = new Box<>();
b.set(10);
```

Bounded types restrict the parameter, e.g. `<T extends Number>` accepts only Number and its subtypes. Wildcards: <?> (any), `<? extends T>`, `<? super T>`.

8.3 List Implementations

A **List** is an ordered collection allowing duplicates and indexed access.

```
import java.util.*;

List<String> fruits = new ArrayList<>();
fruits.add("apple");
fruits.add("banana");
fruits.add(1, "mango");          // insert at index 1
fruits.set(0, "kiwi");           // replace
fruits.remove("banana");
System.out.println(fruits.get(0));      // kiwi
System.out.println(fruits.size());      // 2
System.out.println(fruits.contains("mango"));  // true
for (String f : fruits) System.out.println(f);
```

Feature	ArrayList	LinkedList	Vector
Backed by	resizable array	doubly linked list	resizable array
Random access (get)	Fast O(1)	Slow O(n)	Fast O(1)
Insert/remove middle	Slow O(n)	Fast O(1)*	Slow O(n)
Thread-safe?	No	No	Yes (synchronized)
Best for	reads/iteration	frequent insert/remove	legacy code

**LinkedList insertion is O(1) only once the position is reached.*

8.4 Set Implementations

A **Set** stores *unique* elements (no duplicates). The implementation determines ordering.

```
Set<Integer> hs = new HashSet<>();    // no order, fastest
hs.add(3); hs.add(1); hs.add(3);      // duplicate ignored
System.out.println(hs.size());        // 2

Set<Integer> ts = new TreeSet<>();    // sorted ascending
ts.add(5); ts.add(1); ts.add(3);
System.out.println(ts);               // [1, 3, 5]
```

- **HashSet** — no guaranteed order, O(1) operations, fastest.
- **LinkedHashSet** — preserves *insertion* order.
- **TreeSet** — keeps elements *sorted*; O(log n) operations.

8.5 Map Implementations

A **Map** stores **key→value** pairs with unique keys. It is not a `Collection` but is part of the framework.

```
Map<String, Integer> ages = new HashMap<>();
ages.put("Alice", 30);
ages.put("Bob", 25);
ages.put("Alice", 31);            // overwrites previous value

System.out.println(ages.get("Alice"));            // 31
System.out.println(ages.containsKey("Bob"));      // true
System.out.println(ages.getOrDefault("Eve", 0));  // 0

for (Map.Entry<String, Integer> e : ages.entrySet()) {
    System.out.println(e.getKey() + " = " + e.getValue());
}
```

Implementation	Ordering	Null keys?	Notes
HashMap	none	one null key	general purpose, fastest
LinkedHashMap	insertion order	one null key	predictable iteration
TreeMap	sorted by key	no null key	O(log n), navigable
Hashtable	none	no nulls	legacy, synchronized

8.6 Queue and Deque

A **Queue** typically processes elements first-in-first-out (FIFO). A **Deque** (double-ended queue) allows adding/removing at both ends and can act as a stack (LIFO).

```
Queue<String> q = new LinkedList<>();
q.offer("a"); q.offer("b");      // enqueue at tail
```

```
System.out.println(q.poll());    // a  (dequeue from head)

Deque<Integer> stack = new ArrayDeque<>();
stack.push(1); stack.push(2);    // LIFO
System.out.println(stack.pop()); // 2
```

8.7 Iterator and ListIterator

An **Iterator** traverses a collection and is the only safe way to *remove* elements during iteration. `ListIterator` additionally moves backward and can modify elements.

```
List<Integer> nums = new ArrayList<>(List.of(1, 2, 3, 4));
Iterator<Integer> it = nums.iterator();
while (it.hasNext()) {
    int n = it.next();
    if (n % 2 == 0) it.remove();   // safe removal
}
System.out.println(nums);          // [1, 3]
```

Modifying a collection directly (e.g. with `list.remove()`) during a for-each loop throws `ConcurrentModificationException`.

8.8 Comparable vs Comparator

To sort custom objects you define an ordering. `Comparable` defines a single *natural* order inside the class; `Comparator` defines *external*, reusable orderings.

Aspect	Comparable	Comparator
Package	java.lang	java.util
Method	compareTo(o)	compare(a, b)
Where defined	inside the class	separate / lambda
Orderings	one (natural)	many possible
Sort call	Collections.sort(list)	sort(list, comparator)

```
class Person { String name; int age; /* ctor omitted */ }

// Comparator via lambda - sort by age, then name
list.sort(Comparator.comparingInt((Person p) -> p.age)
                    .thenComparing(p -> p.name));
```

8.9 The Collections Utility Class

`java.util.Collections` offers static helpers that operate on collections.

```
List<Integer> n = new ArrayList<>(List.of(3, 1, 2));
Collections.sort(n);              // [1, 2, 3]
Collections.reverse(n);           // [3, 2, 1]
Collections.shuffle(n);           // random order
int max = Collections.max(n);
int min = Collections.min(n);
```

PART 9 Java 8 Functional Features

9.1 Lambda Expressions

A **lambda expression** is a concise way to represent an anonymous function — a block of behaviour you can pass around. It implements a **functional interface** (one abstract method). Syntax: `(parameters) -> body`.

```
// Old way - anonymous class
Runnable r1 = new Runnable() {
    public void run() { System.out.println("run"); }
};

// Lambda - same thing, far shorter
Runnable r2 = () -> System.out.println("run");

// With parameters
Comparator<Integer> byValue = (a, b) -> a - b;

// Multi-statement body needs braces and return
Comparator<String> byLen = (a, b) -> {
    return a.length() - b.length();
};
```

9.2 Functional Interfaces

A **functional interface** has exactly one abstract method, making it a valid target for a lambda. `java.util.function` ships several general-purpose ones.

Interface	Abstract method	Takes → returns	Purpose
Predicate<T>	test(T)	T → boolean	a condition / filter
Function<T,R>	apply(T)	T → R	transform a value
Consumer<T>	accept(T)	T → void	perform a side effect
Supplier<T>	get()	() → T	produce a value
BiFunction<T,U,R>	apply(T,U)	(T,U) → R	two inputs, one result

```
import java.util.function.*;

Predicate<Integer> isEven = n -> n % 2 == 0;
Function<String, Integer> length = s -> s.length();
Consumer<String> printer = s -> System.out.println(s);
Supplier<Double> random = () -> Math.random();

System.out.println(isEven.test(4));    // true
System.out.println(length.apply("hi")); // 2
printer.accept("hello");
```

9.3 Method References

A **method reference** is shorthand for a lambda that simply calls an existing method, written with `::`.

Kind	Syntax	Equivalent lambda
Static method	`Integer::parseInt`	s -> Integer.parseInt(s)
Instance (particular obj)	`System.out::println`	x -> System.out.println(x)

Kind	Syntax	Equivalent lambda
Instance (any obj of type)	`String::toUpperCase`	s -> s.toUpperCase()
Constructor	`ArrayList::new`	() -> new ArrayList()

9.4 The Stream API

A **Stream** is a sequence of elements supporting functional-style operations (filter, map, reduce). Streams do not store data; they process it through a **pipeline** of *intermediate* operations (lazy, return a stream) ending in one *terminal* operation (produces a result).

```
import java.util.*;
import java.util.stream.*;

List<Integer> nums = List.of(1, 2, 3, 4, 5, 6);

// filter even -> square -> collect to list
List<Integer> result = nums.stream()
        .filter(n -> n % 2 == 0)     // 2, 4, 6
        .map(n -> n * n)             // 4, 16, 36
        .collect(Collectors.toList());
System.out.println(result);          // [4, 16, 36]

int sum = nums.stream()
        .mapToInt(Integer::intValue)
        .sum();                      // 21

long count = nums.stream()
        .filter(n -> n > 3)
        .count();                    // 3
```

Operation	Type	What it does
`filter(pred)`	intermediate	keep elements matching a condition
`map(fn)`	intermediate	transform each element
`sorted()`	intermediate	sort the stream
`distinct()`	intermediate	remove duplicates
`collect(...)`	terminal	gather into a List/Set/Map
`forEach(...)`	terminal	perform an action per element
`reduce(...)`	terminal	combine into a single value
`count()`	terminal	number of elements

9.5 Optional

`Optional<T>` is a container that may or may not hold a value, designed to express “might be absent” explicitly and reduce `NullPointerExceptions`.

```
import java.util.Optional;

Optional<String> maybe = Optional.of("hello");
Optional<String> empty = Optional.empty();

System.out.println(maybe.isPresent());          // true
System.out.println(empty.orElse("default"));    // default
maybe.ifPresent(v -> System.out.println(v));     // hello

String value = empty.orElseGet(() -> "computed");
```

PART 10 Multithreading and Concurrency

10.1 Threads: Process vs Thread

A **process** is an independent running program with its own memory. A **thread** is a lightweight unit of execution *within* a process; threads in the same process share memory, enabling concurrent work. Java supports threads natively.

There are two ways to create a thread:

```
// Way 1: extend Thread
class MyThread extends Thread {
    public void run() {
        System.out.println("Running: " + getName());
    }
}
new MyThread().start();        // start() -> runs run() on a new thread

// Way 2: implement Runnable (preferred - leaves you free to extend)
class Task implements Runnable {
    public void run() {
        System.out.println("Task running");
    }
}
new Thread(new Task()).start();

// With a lambda (Runnable is functional)
new Thread(() -> System.out.println("Lambda thread")).start();
```

Always call `start()`, not `run()`. Calling `run()` directly executes it on the current thread — no new thread is created.

10.2 Thread Life Cycle

A thread moves through several states during its life:

- **NEW** — created but `start()` not yet called.
- **RUNNABLE** — eligible to run; the scheduler may or may not be running it now.
- **BLOCKED** — waiting to acquire a monitor lock.
- **WAITING / TIMED_WAITING** — paused (e.g. via `wait()`, `join()`, `sleep()`).
- **TERMINATED** — finished executing or stopped by an exception.

10.3 Common Thread Methods

Useful control methods on threads:

Method	Effect
`start()`	begins execution in a new thread
`sleep(ms)`	pauses the current thread for ms milliseconds
`join()`	waits for another thread to finish before continuing
`setDaemon(true)`	marks a background thread that dies with the app
`setPriority(n)`	hints scheduling priority (1–10)
`interrupt()`	requests a thread to stop what it is doing

```
Thread t = new Thread(() -> {
    try { Thread.sleep(1000); } catch (InterruptedException e) {}
    System.out.println("done");
});
t.start();
t.join();           // main waits here until t finishes
System.out.println("after join");
```

10.4 Synchronization

When multiple threads modify shared data, results can become corrupted — a **race condition**. `synchronized` ensures only one thread executes a block/method on a given object's lock at a time.

```
class Counter {
    private int count = 0;

    public synchronized void increment() {  // one thread at a time
        count++;
    }
    public int get() { return count; }
}

// Synchronized block - lock only the critical section
synchronized (lockObject) {
    // critical code
}
```

Other tools: `volatile` guarantees visibility of a variable across threads; the `java.util.concurrent` package offers higher-level utilities such as `ReentrantLock`, `AtomicInteger` and `ExecutorService`.

10.5 Inter-thread Communication

`wait()`, `notify()` and `notifyAll()` (all called inside a `synchronized` block) let threads coordinate — the classic producer/consumer pattern.

- `wait()` — releases the lock and pauses until notified.
- `notify()` — wakes one waiting thread.
- `notifyAll()` — wakes all waiting threads.

PART 11 Input / Output and File Handling

11.1 The File Class

`java.io.File` represents a file or directory *path* (not its contents). It is used to test existence, create, delete and inspect files.

```
import java.io.File;

File f = new File("notes.txt");
System.out.println(f.exists());       // true/false
System.out.println(f.getAbsolutePath());
System.out.println(f.length());       // size in bytes

File dir = new File("myFolder");
dir.mkdir();                          // create directory
```

11.2 Byte Streams

Byte streams (`InputStream` / `OutputStream` families) read and write raw bytes — ideal for binary data such as images.

```
import java.io.*;

// Copy a file byte by byte (with auto-close)
try (FileInputStream in = new FileInputStream("src.dat");
     FileOutputStream out = new FileOutputStream("dst.dat")) {
    int b;
    while ((b = in.read()) != -1) {
        out.write(b);
    }
} catch (IOException e) {
    e.printStackTrace();
}
```

11.3 Character Streams

Character streams (`Reader` / `Writer` families) handle text and respect character encoding. Wrapping them in `BufferedReader`/`BufferedWriter` improves performance and adds `readLine()`.

```
import java.io.*;

// Write text
try (BufferedWriter bw = new BufferedWriter(
                            new FileWriter("out.txt"))) {
    bw.write("Hello file");
    bw.newLine();
    bw.write("Second line");
}

// Read text line by line
try (BufferedReader br = new BufferedReader(
                            new FileReader("out.txt"))) {
    String line;
    while ((line = br.readLine()) != null) {
        System.out.println(line);
    }
}
```

11.4 Serialization

Serialization converts an object into a byte stream (to save to disk or send over a network); **deserialization** reconstructs it. The class must implement the marker interface `Serializable`.

```
import java.io.*;

class User implements Serializable {
    private static final long serialVersionUID = 1L;
    String name;
    transient String password;   // 'transient' fields are NOT saved
    User(String n, String p) { name = n; password = p; }
}

// Serialize
try (ObjectOutputStream oos = new ObjectOutputStream(
                                new FileOutputStream("user.ser"))) {
    oos.writeObject(new User("Alice", "secret"));
}

// Deserialize
try (ObjectInputStream ois = new ObjectInputStream(
                                new FileInputStream("user.ser"))) {
    User u = (User) ois.readObject();
    System.out.println(u.name);   // Alice  (password is null)
}
```

transient excludes a field from serialization (use it for passwords or derived data). `serialVersionUID` identifies a class version for compatibility.

11.5 Modern File I/O (java.nio)

The `java.nio.file` API (Java 7+) offers simpler, more powerful file operations through `Files` and `Paths`.

```
import java.nio.file.*;
import java.util.List;

Path path = Paths.get("data.txt");

// Write all lines at once
Files.write(path, List.of("line 1", "line 2"));

// Read all lines into a List
List<String> lines = Files.readAllLines(path);

// Read entire content as one String
String content = Files.readString(path);   // Java 11+
```

PART A Appendix

A.1 Java Keywords (Reserved Words)

These words have special meaning and cannot be used as identifiers:

abstract	assert	boolean	break	byte
case	catch	char	class	const*
continue	default	do	double	else
enum	extends	final	finally	float
for	goto*	if	implements	import
instanceof	int	interface	long	native
new	package	private	protected	public
return	short	static	strictfp	super
switch	synchronized	this	throw	throws
transient	try	void	volatile	while

**const and goto are reserved but unused. true, false and null are literals, not keywords.*

A.2 Operator Precedence (high → low)

When several operators appear in one expression, those higher in this list bind more tightly. Use parentheses to make intent explicit.

Level	Operators	Associativity
Postfix	`expr++ expr--`	left to right
Unary	`++expr --expr + - ! ~`	right to left
Multiplicative	`* / %`	left to right
Additive	`+ -`	left to right
Shift	`<< >> >>>`	left to right
Relational	`< > <= >= instanceof`	left to right
Equality	`== !=`	left to right
Logical AND	`&&`	left to right
Logical OR	`||`	left to right
Ternary	`? :`	right to left
Assignment	`= += -= *= /= %=`	right to left

A.3 Common Pitfalls and Best Practices

Habits that prevent the most frequent Java bugs:

- Compare object content with `.equals()`, never == (which compares references).
- Always override `hashCode()` when you override `equals()`.
- Close resources — prefer **try-with-resources** over manual `finally` blocks.

- Guard against `null`; consider `Optional` for “maybe absent” values.
- Use `StringBuilder` for string building inside loops.
- Program to interfaces: declare variables as `List`/`Map`, not the concrete class.
- Keep fields `private` and expose behaviour, not data (encapsulation).
- Catch specific exceptions, not bare `Exception`; never swallow them silently.
- Prefer `Runnable`/lambdas over extending `Thread`; synchronise access to shared mutable state.

A.4 Quick Glossary

Key terms used throughout this guide:

Term	Meaning
JVM / JRE / JDK	runtime engine / runtime environment / development kit
Bytecode	platform-neutral compiled code the JVM executes
Class / Object	a blueprint / a concrete instance of it
Encapsulation	hiding data behind methods
Inheritance	reusing a parent class via extends
Polymorphism	one interface, many implementations
Abstraction	exposing essentials, hiding detail
Interface	a contract of method signatures
Generics	type parameters for compile-time safety
Collection	a reusable data structure for groups of objects
Lambda	an anonymous function (Java 8+)
Stream	a pipeline for processing sequences of data
Thread	a unit of concurrent execution
Exception	a runtime event that disrupts normal flow

End of guide — keep coding, and revisit the examples by typing them yourself.